МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Комп’ютерний практикум №5

з курсу «Основи розробки програмного забезпечення на платформі Microsoft.NET»

на тему: «Шаблони проектування. Структурні шаблони»

Викладач:

Ліщук К.І.

Виконав:

студент групи ІП-44

Комін Іван

Київ - 2026

Комп‘ютерний практикум № 5. Шаблони проектування. Структурні шаблони

Мета: - ознайомитися з основними шаблонами проектування, навчитися застосовувати їх при проектуванні і розробці ПЗ.

Постановка задачі комп‘ютерного практикуму № 5

При виконанні комп‘ютерного практикуму необхідно виконати наступні дії:

1) Вивчити структурні патерни. Знати загальну характеристику та призначення кожного з них, особливості реалізації кожного з породжуючих патернів та випадки їх застосування.

2) Реалізувати задачу згідно варіанту, запропонованого нижче у вигляді консольного застосування на мові C#. Розробити інтерфейси та класи з застосування одного або декількох патернів. Повністю реалізувати методи, пов‘язані з реалізацією обраного патерну.

3) Повністю описати архітектуру проекту (призначення методів та класів), особливості реалізації обраного патерну. Для кожного патерну необхідно вказати основні класи та їх призначення,

4) Навести UML-діаграму класів

5) Звіт повинен містити:

a. обґрунтування обраного патерну (чому саме він);

b. опис архітектури проекту (призначення методів та класів);

c. UML-діаграму класів

d. особливості реалізації обраного патерну

e. текст програмного коду

f. скріншоти результатів виконання.

Варіант індивідуальних завдань:

8) В системі електронного уряду реалізувати задачу доступу до архівів, котрі зберігаються в офлайн-сховищах. Необхідно реалізувати симуляцію доступу до даних і відкладене підключення до сховища.

Опис архітектури проєкту:

1. Обґрунтування обраного патерна

Для реалізації системи доступу до офлайн-архівів електронного уряду обрано структурний патерн **Proxy (Заступник)**, а саме його варіацію — **Virtual Proxy**.

Чому саме цей патерн:

- **Відкладена ініціалізація (Lazy Loading):** Офлайн-сховища є «важкими» ресурсами, що потребують часу на фізичне підключення носіїв. Патерн дозволяє не створювати об'єкт реального з'єднання, доки користувач фактично не запитає вміст документа.
- **Економія ресурсів:** Система не витрачає час та потужності на ініціалізацію з'єднання, якщо користувач лише переглядає метадані або списки доступних архівів.
- **Додатковий контроль:** Заступник дозволяє легко впровадити логування запитів або перевірку прав доступу, не вносячи змін у логіку роботи самого сховища.
- **Прозорість для клієнта:** Клієнтський код працює з проксі-об'єктом через інтерфейс, тому він може навіть не знати, що завантаження даних відбувається із затримкою через офлайн-статус.

2. Опис архітектури проєкту

Архітектура базується на створенні об'єкта-посередника, який контролює доступ до реального сервісу архівації.

Основні класи та їх призначення:

- **IArchiveService (Subject):** Спільний інтерфейс, який визначає набір методів для роботи з архівами. Він забезпечує взаємозамінність реального сервісу та його заступника.
- **RealOfflineArchive (Real Subject):** Клас, що представляє реальне сховище. Його створення імітує повільне фізичне підключення до носіїв інформації.
- **ArchiveProxy (Proxy):** Клас-посередник, який реалізує ту саму логіку інтерфейсу, але відкладає створення реального об'єкта до моменту першого звернення за даними.
- **AccessLog:** Внутрішня структура проксі-класу для ведення історії звернень до системи.

3. Призначення методів

Клас ArchiveProxy (Ключові методи):

- **GetDocument(string id):** Основний метод доступу. Він перевіряє, чи створено екземпляр RealOfflineArchive. Якщо ні — ініціює підключення, після чого делегує запит реальному об'єкту.
- **ShowAccessHistory():** Метод для виведення журналу звернень, який ведеться заступником незалежно від стану підключення сховища.

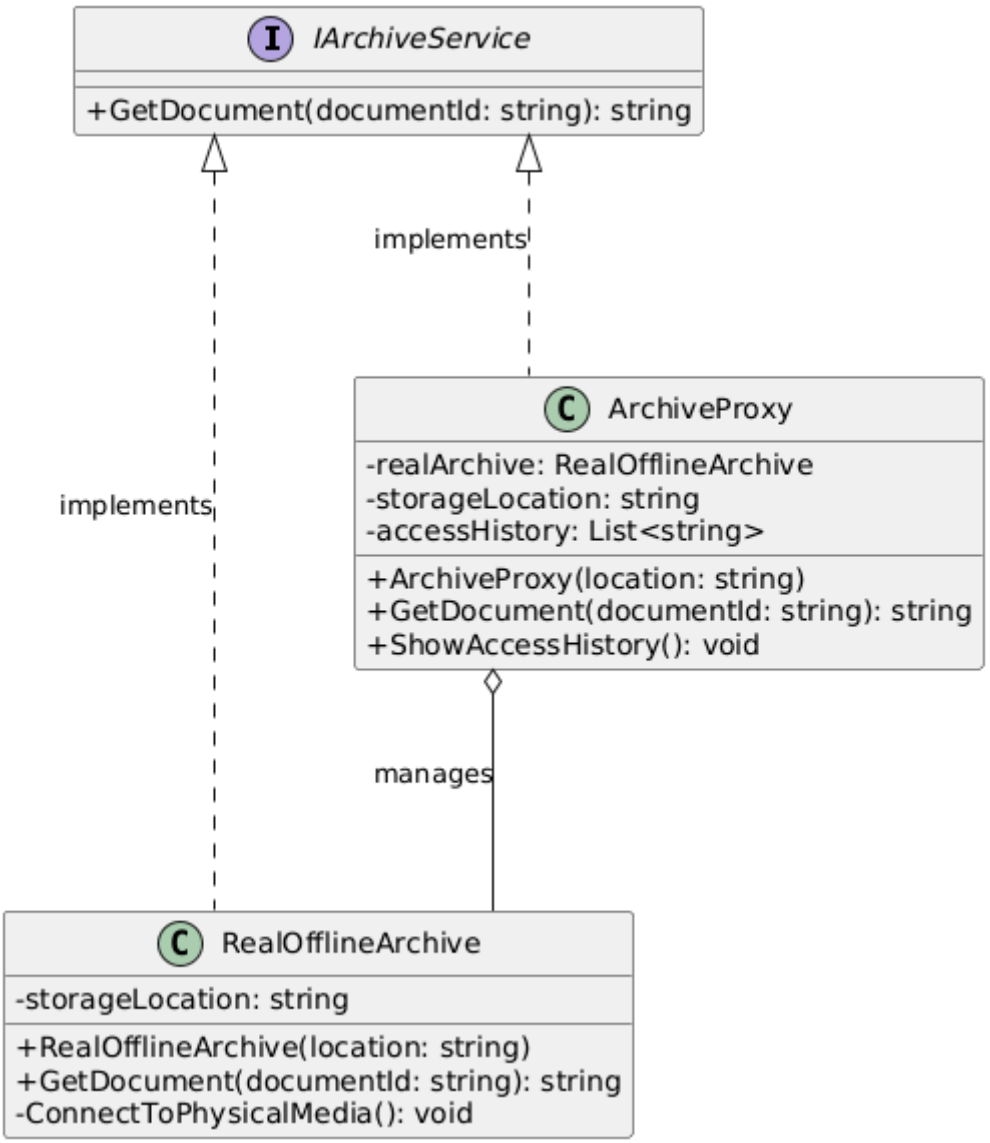
Клас RealOfflineArchive:

- **ConnectToPhysicalMedia():** Приватний метод, що симулює технічну затримку (за допомогою Thread.Sleep), необхідну для переведення сховища в онлайн-режим.
- **GetDocument(string id):** Повертає безпосередній вміст архівного документа після успішного підключення.

4. Особливості реалізації патерна

- **Lazy Initialization (Відкладене створення):** Об'єкт RealOfflineArchive ініціалізується лише всередині методу GetDocument і тільки за потреби. Це ключова особливість «віртуального заступника».
- **Симуляція затримки:** Використання програмної паузи дозволяє наочно продемонструвати різницю між першим (повільним) зверненням та наступними (миттєвими) запитами до вже активного сховища.
- **Ізоляція логіки логування:** Ведення історії запитів винесене в проксі-клас, що дозволяє реальному сервісу виконувати лише свою пряму функцію — видачу даних.
- **Кешування з'єднання:** Після того, як проксі один раз створив реальний об'єкт сховища, він зберігає посилання на нього, забезпечуючи швидкий доступ до всіх наступних документів у межах сесії.

UML-діаграма класів



Текст програмного коду

```
// 1. Subject Interface
public interface IArchiveService
{
    string GetDocument(string documentId);
}

// 2. Real Subject (Offline Storage)
public class RealOfflineArchive : IArchiveService
{
    private readonly string _storageLocation;

    public RealOfflineArchive(string location)
    {
        _storageLocation = location;
        ConnectToPhysicalMedia(); // Simulated expensive operation
```

```
    }

    private void ConnectToPhysicalMedia()
    {
        Console.WriteLine(
            $"[System]: Initializing physical connection to storage at
{_storageLocation}..."
        );
        // Simulate 3 seconds delay for mechanical spin-up or robotic arm
        Thread.Sleep(3000);
        Console.WriteLine("[System]: Offline storage is now ONLINE.");
    }

    public string GetDocument(string documentId)
    {
        return $"Content of document '{documentId}' retrieved from
high-security offline vault at {_storageLocation}.";
    }
}

// 3. Proxy (Virtual Proxy for Delayed Connection)
public class ArchiveProxy(string location) : IArchiveService
{
    private RealOfflineArchive? _realArchive;
    private readonly string _storageLocation = location;
    private readonly List<string> _accessHistory = [];

    public string GetDocument(string documentId)
    {
        // Logging access even before connection
        _accessHistory.Add($"{DateTime.Now}: Requested {documentId}");
        Console.WriteLine($"[Proxy]: Request received for ID: {documentId}.
Processing...");

        // Delayed connection (Lazy Initialization)
        if (_realArchive == null)
        {
            Console.WriteLine(
                "[Proxy]: Offline vault is not connected. Initiating
delayed connection..."
            );
            _realArchive = new RealOfflineArchive(_storageLocation);
        }
        else
        {
```

```
            Console.WriteLine("[Proxy]: Using existing connection to
vault.");
        }

        return _realArchive.GetDocument(documentId);
    }

    public void ShowAccessHistory()
    {
        Console.WriteLine("\n\tACCESS HISTORY LOG:");
        foreach (var log in _accessHistory)
            Console.WriteLine(log);
    }
}

// 4. Main Program
class Program
{
    static void Main()
    {
        const string vaultLocation = "Vault-Sector-7G";
        // Client works with the Proxy instead of the Real Subject
        IArchiveService archiveAccess = new ArchiveProxy(vaultLocation);

        Console.WriteLine("Step 1: Browsing metadata (no connection needed
yet)...");
        // Imagine metadata is stored in a fast DB, but actual content is
offline.

        Console.WriteLine("\nStep 2: Requesting Document #101...");
        // This will trigger the slow connection
        string doc1 = archiveAccess.GetDocument("DOC-101-SECRET");
        Console.WriteLine($"RESULT: {doc1}");

        Console.WriteLine("\nStep 3: Requesting Document #102...");
        // This will be fast as the connection is already established
        string doc2 = archiveAccess.GetDocument("DOC-102-PUBLIC");
        Console.WriteLine($"RESULT: {doc2}");

        ((ArchiveProxy)archiveAccess).ShowAccessHistory();
    }
}
```

Скріншоти результатів виконання

```
/home/ivkins/.vscode/extensions/ms-dotnettools.csharp-2.130.5-linux-x64/.debugger/vsdbg --interpreter=vsco
Step 1: Browsing metadata (no connection needed yet)...

Step 2: Requesting Document #101...
[Proxy]: Request received for ID: DOC-101-SECRET. Processing...
[Proxy]: Offline vault is not connected. Initiating delayed connection...
[System]: Initializing physical connection to storage at Vault-Sector-7G...
[System]: Offline storage is now ONLINE.
RESULT: Content of document 'DOC-101-SECRET' retrieved from high-security offline vault at Vault-Sector-7G.

Step 3: Requesting Document #102...
[Proxy]: Request received for ID: DOC-102-PUBLIC. Processing...
[Proxy]: Using existing connection to vault.
RESULT: Content of document 'DOC-102-PUBLIC' retrieved from high-security offline vault at Vault-Sector-7G.

        ACCESS HISTORY LOG:
5/4/2026 8:24:37 PM: Requested DOC-101-SECRET
5/4/2026 8:24:40 PM: Requested DOC-102-PUBLIC
~/Documents/dotnet-labs/lab5   master !28 ?1
```

Висновок: У ході виконання комп’ютерного практикуму було досліджено та практично реалізовано структурний патерн проектування **Proxy (Заступник)** для вирішення задачі доступу до офлайн-архівів у системі електронного уряду. Використання варіації **Virtual Proxy** дозволило впровадити механізм відкладеної ініціалізації (lazy loading), що забезпечило суттєву оптимізацію ресурсів системи за рахунок підключення «важких» об’єктів сховища лише у момент фактичного запиту даних. Реалізована архітектура забезпечила повну прозорість доступу для клієнтського коду через спільний інтерфейс, дозволила інтегрувати систему логування звернень та успішно симулювати фізичні затримки, характерні для роботи з механічними офлайн-носіями. Таким чином, було доведено ефективність застосування структурних патернів для побудови складних систем із контрольованим доступом до зовнішніх ресурсів, що мають високу часову інертність відгуку.